Universidad Nacional de San Martín
Escuela de Ciencia y Tecnología (ECyT)
Licenciatura en Desarrollo de Software · Verificación y Validación de Software

Trabajo Práctico Integrador

ATALAYA 3.1.0

Estrategia de testing e informe de QA

Grupo E · Turno noche
Roman Cardozo
Valentino Geronzi
Gonzalo Ventrici
Nicolas Beda

Docentes: Ing. Maximiliano Di Ludovico · Ing. Alejandro Millán
Fecha de entrega: 13/11/2026 · Versión 7 del informe

Contenido

1. Introducción

2. Sistema bajo prueba y alcance

3. Fundamentos aplicados

4. Análisis de riesgos

5. Revisión de requerimientos

6. Plan de pruebas

7. Diseño de pruebas

8. Casos de prueba

9. Criterios de aceptación y BDD

10. Automatización, TDD, API e interfaz

11. Defectos, Definition of Done y métricas

12. Informe final

13. Uso de inteligencia artificial

14. Entrega

Cambios respecto de la versión 6

Sección	Cambio
2, 3, 7, 10, 12	El panel web y las pruebas con Selenium pasan a estar dentro del alcance, como pidió el profesor el 30/10.
5, 11	Sin Jira: los defectos se llevan en `docs/defectos.md` y en la hoja Defectos de la planilla. Se suman las decisiones de implementación (ATA-8 a ATA-12).
6	Entorno, herramientas, cronograma y forma de trabajo ajustados a lo que hicimos.
7.3	El código de `ClasificadorNivel` es el real (`BigDecimal`).
8	Casos nuevos CP-37 a CP-41 y UI-01 a UI-03 (44 en total).
10, 12	Resultados de la ejecución, cobertura, mutación y defectos.

1. Introducción

ATALAYA es un sistema de asistencia a la operación de una central nuclear: recibe señales de planta, estima la probabilidad de distintas desviaciones y genera alertas para la sala de control. No tiene ningún camino de escritura hacia los sistemas de control ni de protección, así que el riesgo que nos importa es que el operador tome una mala decisión por una alerta falsa, una alerta que no llega o que llega tarde, o por exceso de alertas.

El enunciado no trae código y los lineamientos piden entregar el SISTEMA en Java 21 con Spring Boot. Por eso implementamos una versión reducida de ATALAYA que cubre la lógica de mayor riesgo y, sobre ella, planificamos, diseñamos y ejecutamos las pruebas. Como confirmó el profesor, la complejidad del código no es el objetivo: lo mantuvimos chico para que las pruebas sean lo importante.

2. Sistema bajo prueba y alcance

El SISTEMA es una API REST con Spring Boot, persistencia en H2 y autenticación por roles con Spring Security, más un panel web mínimo para la sala de control. Las estimaciones del modelo de IA no se calculan: en el perfil qa se inyectan por un endpoint, igual que el avance del reloj, que es un `java.time.Clock` inyectado en las clases que manejan tiempos. Así podemos probar 45 s o 3 min sin esperar.

Incluido en el alcance	Fuera del alcance y motivo
RF-01/02: validación de rangos, duplicados y orden de llegada	RF-04/05 modo operativo y calibración: menor riesgo para la lógica de alertas
RF-03: redundancia de tres canales	RF-10 modo degradado y RF-11 integraciones y auditoría: dependen de sistemas externos
RF-06: niveles de riesgo y factores	Modelo de IA real y su evaluación estadística: no disponible
RF-07/08: alertas, repetición y escalamiento	Desempeño, disponibilidad 99,95 % y carga: requieren infraestructura real
RF-09: agrupación en maniobras (nuevo en 3.1.0)	Red segregada, licenciamiento y seguridad nuclear
RF-12: umbrales con doble aprobación	
Seguridad: autenticación y permisos por rol	
Panel web: alertas, reconocimiento y estado de canales (pruebas con Selenium)	

Endpoint	Uso
POST /api/v1/lecturas	Ingreso de una lectura (magnitud, canal, valor, secuencia)
GET /api/v1/magnitudes/{magnitud}	Valor consolidado, si es utilizable, estado de cada canal y avisos
GET /api/v1/estimaciones/actual	Probabilidad, nivel y factores por tipo de estimación
GET /api/v1/alertas · POST /api/v1/alertas/{id}/reconocimiento	Consultar y reconocer alertas
POST /api/v1/maniobras · DELETE /api/v1/maniobras/{id}	Declarar y finalizar maniobras
GET y PUT /api/v1/config/umbrales · POST …/{id}/aprobacion	Ver los umbrales vigentes, solicitar y aprobar cambios
GET /api/v1/version	Versión de software (3.1.0) y del modelo
POST /api/v1/qa/estimaciones · /qa/reloj · /qa/reset	Solo perfil qa: inyectar estimaciones, avanzar el reloj y volver a la precondición base

Endpoint	Uso
/login · /panel	Panel web con login por formulario

3. Fundamentos aplicados

Verificamos con revisión de requerimientos, revisión de código y pruebas contra cada RF. Validamos con criterios de aceptación escritos para que operación y seguridad puedan leerlos (sección 9). En el informe usamos error para la equivocación de una persona, defecto para lo que queda mal en un artefacto y falla para lo que se observa al ejecutar. Por ejemplo: interpretar “más del 2 %” como “2 % o más” es un error; escribir >= en vez de > en el validador es el defecto; que un canal al 2,00 % quede como no confiable es la falla (CP-08).

Así aplicamos los principios del testing en este trabajo:

Principio	Cómo lo aplicamos
Las pruebas muestran la presencia de defectos, no su ausencia	En el informe final decimos qué no pudimos verificar
No se puede probar todo	Valores límite, partición de equivalencia y pairwise
Probar temprano	Revisamos los requerimientos antes de codificar (sección 5)
Los defectos se agrupan	Más casos en redundancia y escalamiento, donde están los bordes
Paradoja del pesticida	Revisamos los mutantes que sobreviven y agregamos casos
Las pruebas dependen del contexto	La severidad se mide por el efecto sobre la decisión del operador

Las características de calidad de ISO/IEC 25010 que priorizamos para este SISTEMA son las siguientes:

Característica	Subcaracterísticas	Prioridad	Cómo la evaluamos
Adecuación funcional	Corrección, completitud	Alta	Casos de caja negra y blanca sobre los RF del alcance
Fiabilidad	Tolerancia a fallos, madurez	Alta	Canales sin actualizar, duplicados, lecturas fuera de orden
Seguridad	Autenticidad, confidencialidad, responsabilidad	Alta	Permisos por rol, registro de quién reconoce, endpoints de QA apagados en producción
Mantenibilidad	Capacidad de ser probado, modificabilidad	Media	Reloj inyectable, cobertura y mutación
Usabilidad	Operabilidad	Media	Selenium sobre el panel: reconocer una alerta en 2 interacciones o menos y ver el estado de los canales (UI-01 a UI-03)
Eficiencia de desempeño	Comportamiento temporal	Baja en este alcance	Requiere infraestructura real; queda fuera

No usamos una herramienta de análisis estático. La revisión de código la hicimos con esta checklist antes de integrar cada cambio:

Punto de revisión	Por qué
Los bordes usan el operador que pide el requerimiento (> o >=)	Es el defecto más probable en RF-03, RF-06 y RF-08
Las clases con tiempos reciben el Clock y no llaman a now() directo	Si no, las pruebas de tiempo no son repetibles
Las comparaciones de las reglas usan BigDecimal	Con double, 306,60 − 300,00 da 6,6000000000000227 y el caso del 2,00 % falla
Se validan nulos y valores fuera de rango en la entrada	RF-01/02

Punto de revisión	Por qué
Cada prueba tiene comentario con su propósito y el ID del caso	Lo piden los lineamientos y ayuda a la trazabilidad
Los endpoints de QA están anotados con el perfil qa	Evita que lleguen a producción (CP-36)

4. Análisis de riesgos

Priorizamos por severidad (efecto sobre la decisión del operador) y probabilidad.

ID	Riesgo	Sev.	Prob.	Cómo lo cubrimos
R1	Alerta crítica que no llega o llega tarde	Alta	Media	Bordes de nivel y de escalamiento (CP-13, CP-21)
R2	Exceso de alertas repetidas: el operador deja de prestarles atención	Alta	Alta	Repetición de 3 min y agrupación en maniobras (CP-19, CP-27)
R3	La agrupación de maniobras oculta un desvío real	Alta	Media	Bordes de la banda, maniobra con falla de instrumento, expiración, pairwise
R4	Se usa un canal no confiable para estimar	Alta	Media	Tabla de decisión de RF-03
R5	Lecturas duplicadas, desordenadas o fuera de rango	Alta	Alta	CP-01 a CP-05
R6	Escalamiento a destiempo o duplicado	Alta	Media	Reloj simulado y caja blanca (CP-21 a CP-26)
R7	Cambio de umbral sin la aprobación requerida	Alta	Baja	CP-32, CP-33 y CP-40
R8	Acceso con un rol indebido	Media	Baja	CP-34 a CP-36, también contra el servidor real
R9	El panel muestra algo distinto de lo que informa la API	Media	Baja	UI-02 y UI-03
R10	Datos sintéticos poco representativos	Media	Alta	Se declara como limitación

5. Revisión de requerimientos

La revisión de requerimientos fue la primera prueba que hicimos. Cada ambigüedad quedó registrada como defecto de especificación (sección 11). Hasta tener respuesta, el SISTEMA implementa la decisión de la tabla.

#	Ambigüedad	Decisión	Defecto
A1	“Discrepa más del 2 % del rango respecto de los otros”: ¿contra qué canal o valor?	Contra la mediana de los canales con valor	ATA-1
A2	“2 % del rango”: ¿de qué rango?	Rango físico de RF-01 (330 °C → 6,6 °C)	ATA-1
A3	Al recuperarse un canal, ¿cuántas lecturas buenas hacen falta?	3 lecturas seguidas	ATA-2
A4	“Salvo que empeore”: ¿cambio de nivel o aumento de probabilidad?	Cambio a un nivel mayor	ATA-3
A5	Dos reconocimientos simultáneos de la misma alerta	Vale el primero; el segundo recibe 409	ATA-4
A6	Maniobra declarada que nunca se ejecuta	Expira a los 30 min	ATA-5
A7	Con alto en 30 %, ¿atención pasa a ser de 15 % a 30 %?	Sí	ATA-6
A8	El umbral crítico no tiene máximo	Máximo 100 %	ATA-7

Al implementar aparecieron otras cinco preguntas que el enunciado no contesta. Las resolvimos así:

Tema	Decisión	Defecto
RF-03, regla 1: ¿“sin actualizar” mira la última lectura aceptada o la recibida?	La recibida (aceptada o descartada por rango). Si no, un canal que envía 351 °C quedaría SIN_ACTUALIZACION y nunca FUERA_DE_RANGO (CP-10). Las duplicadas y fuera de orden no cuentan	ATA-8

Tema	Decisión	Defecto
RF-07: nivel menor dentro de los 3 min	No se emite: bajar de nivel no es empeorar	ATA-9
RF-09: ¿puede haber más de una maniobra activa?	No. Declarar otra devuelve 409. La alerta informativa es una por maniobra y agrupa estimaciones de cualquier tipo	ATA-10
RF-12: la regla “alto menor que crítico”	Nunca puede fallar (alto ≤ 50 < 65 ≤ crítico). Queda en el código porque la pide el requerimiento	ATA-11
RF-12: “aprueba el mismo usuario”	Con un rol por usuario, por la API el supervisor recibe 403 antes de llegar a esa regla. El 409 se prueba sobre el servicio	ATA-12

Otras decisiones menores: una lectura se clasifica en el orden duplicada, fuera de orden, fuera de rango y aceptada; el valor consolidado se redondea a 2 decimales; la mediana de dos valores es su promedio; el nombre de la magnitud en la ruta no distingue mayúsculas; y los endpoints de QA no piden credenciales, para que con el perfil prod respondan 404 y no 401 (CP-36).

6. Plan de pruebas

El objetivo es reunir evidencia suficiente sobre la lógica de mayor riesgo para poder recomendar si se libera o no. Probamos lo incluido en la sección 2. La estrategia es basada en riesgos: primero redundancia, niveles y escalamiento, después maniobras, configuración y la interfaz.

No probamos lo que quedó fuera del alcance en la sección 2, ni el modelo de IA. Sobre el modelo, el requerimiento pide una sensibilidad mínima del 95 %, y con pocos eventos críticos ese número es engañoso: 19 detecciones sobre 20 dan 95 %, pero el intervalo de confianza va aproximadamente de 75 % a 99,9 %. Para afirmar que la sensibilidad es de al menos 95 % con 95 % de confianza harían falta unos 59 eventos detectados sin ninguna falla. Lo dejamos planteado como método para cuando el modelo esté disponible.

Para empezar a ejecutar pedimos que el proyecto compile con `mvn verify`, que exista el perfil qa y que los casos estén revisados por otro integrante. Para terminar, que todos los casos estén ejecutados, que no haya defectos críticos abiertos y que la cobertura y la mutación superen lo definido en la sección 11. Suspendemos si el build falla o si un defecto bloquea el resto de las pruebas, y retomamos repitiendo los casos afectados.

Entorno	Uso
Desarrollo	Cada integrante con su IDE (IntelliJ IDEA o VS Code) y JDK 21; pruebas unitarias y TDD
QA	Perfil qa con reloj y estimaciones inyectables; suite completa, Postman y Selenium
Staging	Perfil prod; se verifica que los endpoints de QA no existan (CP-36)
Producción	Sala de control; fuera del alcance del trabajo

Integrante	Rol
Roman Cardozo	Líder de QA; lecturas y redundancia (CP-01 a CP-11)
Valentino Geronzi	Niveles y escalamiento (CP-12 a CP-22)
Gonzalo Ventrici	Maniobras, pairwise y umbrales (CP-23 a CP-33)
Nicolas Beda	Seguridad e interfaz con Selenium (CP-34 a CP-41, UI-01 a UI-03)

Así distribuimos el trabajo:

Fecha	Actividad	Esfuerzo
30/10	Revisión con los docentes y cierre de alcance; revisión de requerimientos	4 h

Fecha	Actividad	Esfuerzo
31/10	Proyecto Maven, RF-01 a RF-12 con sus pruebas; TDD en ValidadorRedundancia y ServicioEscalamiento; Cucumber, Selenium y colección de Postman	24 h
01/11 al 08/11	Revisión de casos, análisis de mutantes, ajustes de la colección y del panel	24 h
09/11	Ejecución completa con evidencia, registro de defectos y planilla	12 h
10/11 al 12/11	Informe final y ensayo de la exposición	8 h
13/11	Entrega en el campus	1 h

Herramientas que usamos:

Herramienta	Uso
Java 21, Spring Boot 3, Maven (wrapper incluido)	SISTEMA y ejecución de todas las pruebas
JUnit 5, Spring Boot Test, MockMvc, Mockito	Pruebas unitarias, de integración y de API; mocks para aislar servicios
Cucumber	Criterios de aceptación en Dado-Cuando-Entonces
Selenium (HtmlUnit y Chrome)	Pruebas de interfaz sobre el panel
JaCoCo y PIT	Cobertura y mutación
Postman y newman	Colección de la API y su ejecución por consola
Makefile	Un comando por tipo de prueba, evidencias y armado de la entrega
Planilla y docs/defectos.md	Casos, trazabilidad, resultados y registro de defectos (no usamos Jira)

Los cambios de alcance o de requerimientos se registran como defecto de especificación, se revisa qué casos afectan en la matriz de trazabilidad y se vuelve a correr la regresión completa. Trabajamos sobre una rama principal y no integramos nada con el build en rojo: `mvn verify` tiene que pasar antes de cada commit.

7. Diseño de pruebas

Probamos en cinco niveles. Unitario: las clases de dominio con JUnit 5, y los servicios aislados con Mockito. Integración: los servicios con H2, usando Spring Boot Test. Sistema: la API con MockMvc, contra el servidor real y con Postman. Interfaz: el panel con Selenium. Aceptación: escenarios en Cucumber. Las técnicas de caja negra se aplican donde hay reglas con bordes claros, la caja gris donde importa lo que queda guardado en la base, y la caja blanca en las dos clases con más decisiones. No hacemos combinatoria completa (serían 162 casos) ni esperas en tiempo real, que son lentas y frágiles.

7.0 Casos de uso

Los casos de prueba salen de estos casos de uso. En cada uno indicamos el flujo principal y los alternativos que probamos.

CU	Actor	Flujo principal	Flujos alternativos
CU-01 Ingresar lecturas (RF-01 a RF-03)	Sistema de adquisición	Envía una lectura válida de un canal; el SISTEMA la guarda y recalcula el estado de los canales	Valor fuera de rango, mensaje inválido, duplicado, fuera de orden, canal que discrepa o deja de actualizarse
CU-02 Consultar nivel de riesgo (RF-06)	Operador de reactor	Consulta la estimación vigente y ve el nivel y, desde atención, los tres factores principales	Probabilidad inválida; umbrales ajustados

CU	Actor	Flujo principal	Flujos alternativos
CU-03 Reconocer y escalar alertas (RF-07/08)	Operador, jefe de turno, supervisor de SO	Recibe la alerta en el panel y la reconoce dentro del plazo	No la reconoce y escala; la alerta empeora; se repite; dos reconocimientos a la vez; alerta inexistente
CU-04 Declarar una maniobra (RF-09)	Jefe de turno	Declara la maniobra con su banda; las variaciones dentro de la banda se agrupan	Desvío fuera de banda; falla de un canal; la maniobra no se ejecuta y expira; segunda maniobra
CU-05 Cambiar umbrales (RF-12)	Supervisor de SO y jefe de turno	El supervisor solicita, el jefe aprueba y el umbral cambia	Valor fuera de límites; falta la segunda aprobación; aprueba el mismo usuario; se aprueba dos veces
CU-06 Acceder al SISTEMA	Todos los roles	Se autentica y opera según su rol	Sin credenciales; rol sin permiso; endpoints de QA en producción

Trazabilidad entre requerimientos y casos (la matriz completa, con resultados y defectos, está en la planilla):

Requerimiento	Casos
RF-01/02	CP-01 a CP-05
RF-03	CP-06 a CP-12, CP-29, CP-31, CP-37, UI-03
RF-06	CP-13 a CP-17, UI-02
RF-07/08	CP-18 a CP-26, CP-31, CP-38, UI-01
RF-09	CP-27 a CP-31, CP-39
RF-12	CP-16, CP-32, CP-33, CP-40
RNF seguridad	CP-34 a CP-36
RNF usabilidad	UI-01 a UI-03
Versión de software	CP-41

7.1 Tabla de decisión del estado de un canal (RF-03)

Condición	R1	R2	R3	R4
Recibió una lectura en los últimos 10 s	S	N	S	S
La última lectura recibida está dentro del rango físico	S	-	N	S
Discrepancia con la mediana ≤ 2 % del rango	S	-	-	N
Estado	Confiable	No confiable (SIN_ACTUALIZACION)	No confiable (FUERA_DE_RANGO)	No confiable (DISCREPANCIA)
Caso	CP-06	CP-09	CP-10	CP-07

Las reglas se evalúan en ese orden. Un canal no confiable vuelve a ser confiable después de 3 lecturas seguidas que cumplen las tres condiciones (A3, CP-12).

7.2 Estados de una alerta alta o crítica

Estado	Evento	Siguiente	Caso
Emitida	Reconocimiento	Reconocida	CP-23
Emitida	Vence el plazo (45 s o 90 s)	Escalada	CP-21, CP-22
Escalada	Reconocimiento	Reconocida	CP-31

Estado	Evento	Siguiente	Caso
Emitida o escalada	Sube de nivel	Nueva alerta del nivel mayor	CP-20
Reconocida	Reconocimiento	Sin cambio (409)	CP-24

7.3 Caja blanca

Para ClasificadorNivel tomamos el código y calculamos la complejidad ciclomática: cuatro decisiones, V(G) = 5. Elegimos un dato por cada camino básico y agregamos p = 100,1 para cubrir la segunda condición de D1 (CP-17). Con ServicioEscalamiento hicimos lo mismo apuntando a cobertura de decisiones (CP-26). Las dos clases quedaron con 100 % de sentencias y ramas en JaCoCo.

```
public NivelRiesgo clasificar(BigDecimal p, Umbrales u) {
    Objects.requireNonNull(p, "La probabilidad es obligatoria");
    if (p.compareTo(minimo) < 0 || p.compareTo(maximo) > 0) {   // D1
        throw new ProbabilidadInvalidaException(p);
    }
    if (p.compareTo(u.critico()) >= 0) {                        // D2
        return NivelRiesgo.CRITICO;
    }
    if (p.compareTo(u.alto()) >= 0) {                           // D3
        return NivelRiesgo.ALTO;
    }
    if (p.compareTo(inicioAtencion) >= 0) {                     // D4
        return NivelRiesgo.ATENCION;
    }
    return NivelRiesgo.NORMAL;
}
```

`minimo`, `maximo` e `inicioAtencion` son 0, 100 y 15. Están como campos de instancia y no como constantes `static final` para que PIT pueda mutarlos (sección 10.2).

Camino	Recorrido	Dato	Resultado
C1	D1 verdadera	−1	Excepción
C2	D2 verdadera	80	CRITICO
C3	D3 verdadera	50	ALTO
C4	D4 verdadera	20	ATENCION
C5	Todas falsas	5	NORMAL

7.4 Caja gris

Verificamos en la base H2 que las lecturas duplicadas no se guarden dos veces (CP-04), que el reconocimiento quede con usuario y hora (CP-23) y que dos reconocimientos simultáneos no dejen dos registros (CP-25). También revisamos la estructura de las respuestas JSON y que los endpoints de QA no existan con el perfil de producción (CP-36).

7.5 Pairwise

En la generación y el escalamiento de alertas se cruzan cinco parámetros: maniobra (3 valores), canales (3), rol que reconoce (3), momento del reconocimiento (3) y nivel (2). Armamos 10 filas a mano en vez de 162 y comprobamos con un script que cubren los 78 pares posibles entre los cinco parámetros. Las filas están en `src/test/resources/datos/pairwise.csv` y las ejecuta CP-31.

Fila	Maniobra	Canales	Reconoce	Cuándo	Nivel	Esperado
1	Ninguna	3 confiables	Operador	Antes	Alto	Reconocida sin escalar
2	Ninguna	1 no confiable	Jefe	Después	Alto	Aviso; escala a 90 s; reconocida
3	Ninguna	Menos de 2	Superv. SO	Nunca	Alto	Aviso; escala a 90 s; queda escalada
4	Expirada	3 confiables	Jefe	Nunca	Alto	Escala a 90 s; queda escalada
5	Expirada	1 no confiable	Superv. SO	Antes	Alto	Aviso; reconocida sin escalar
6	Expirada	Menos de 2	Operador	Después	Crítico	Aviso; escala a 45 s; reconocida
7	Activa	3 confiables	Superv. SO	Después	Alto	No se agrupa; escala a 90 s; reconocida
8	Activa	1 no confiable	Operador	Nunca	Crítico	Aviso aparte; escala a 45 s; queda escalada
9	Activa	Menos de 2	Jefe	Antes	Crítico	Aviso; no se agrupa; reconocida sin escalar
10	Ninguna	3 confiables	Superv. SO	Antes	Crítico	Reconocida sin escalar

“Antes” es a los 30 s de emitida; “Después”, 5 s pasado el plazo de escalamiento. En maniobra activa, el valor de potencia queda fuera de la banda, por eso la alerta no se agrupa.

7.6 Permisos por rol

Operación	Adquisición	Operador	Jefe de turno	Superv. SO	Auditor
Enviar lecturas	Sí	No	No	No	No
Consultar magnitudes, estimaciones y alertas	No	Sí	Sí	Sí	Sí
Reconocer alertas	No	Sí	Sí	Sí	No
Declarar o finalizar maniobra	No	No	Sí	No	No
Solicitar cambio de umbral	No	No	No	Sí	No
Aprobar cambio de umbral	No	No	Sí	No	No

Sin credenciales, 401. Con un rol no permitido, 403. Lo que no figura en la matriz se niega; `GET /version` lo puede usar cualquier usuario autenticado.

8. Casos de prueba

Diseñamos 44 casos: los 36 originales, CP-37 a CP-41 (agregados al revisar que cada prueba automatizada tuviera su caso) y UI-01 a UI-03 para la interfaz. En la planilla Matriz_Trazabilidad_ATALAYA.xlsx cada uno tiene identificador, requerimiento, precondiciones, datos de prueba, pasos, resultado esperado y criterio de aprobación, más el resultado, la evidencia, el defecto, quién lo ejecutó y la fecha. La precondición “Base” es: SISTEMA levantado con perfil qa, base H2 limpia, reloj de prueba en T0, sin maniobra activa, umbrales por defecto (alto 40 %, crítico 75 %) y usuario autenticado con el rol indicado. Los datos son todos sintéticos.

ID	Req.	Caso	Técnica	Caja	Ejecución
CP-01	RF-01/02	Límites de temperatura del primario	Valores límite	Negra	Postman + JUnit
CP-02	RF-01/02	Límites de presión, presurizador, potencia y dosis	Valores límite	Negra	JUnit (parametrizado)
CP-03	RF-01/02	Mensajes inválidos	Partición de equivalencia	Negra	Postman + MockMvc
CP-04	RF-01/02	Lectura duplicada	Adivinanza de errores	Gris	JUnit + consulta a H2
CP-05	RF-01/02	Lectura fuera de orden	Adivinanza de errores	Negra	JUnit
CP-06	RF-03	Tres canales coherentes	Tabla de decisión (R1)	Negra	JUnit

ID	Req.	Caso	Técnica	Caja	Ejecución
CP-07	RF-03	Canal que discrepa 2,01 %	Tabla de decisión (R4) + valores límite	Negra	Postman + JUnit + Cucumber
CP-08	RF-03	Discrepancia de exactamente 2,00 %	Valores límite	Negra	JUnit + Cucumber
CP-09	RF-03	Canales sin actualizar 10 s y 10,1 s	Tabla de decisión (R2) + límites de tiempo	Negra	JUnit (reloj simulado)
CP-10	RF-03	Valor fuera de rango en un canal	Tabla de decisión (R3)	Negra	JUnit
CP-11	RF-03	Menos de dos canales confiables	Tabla de decisión	Negra	Postman + JUnit
CP-12	RF-03	Recuperación de un canal	Transición de estados	Negra	JUnit
CP-13	RF-06	Bordes de nivel	Valores límite	Negra	Postman + JUnit
CP-14	RF-06	Probabilidad inválida	Partición de equivalencia	Negra	JUnit
CP-15	RF-06	Tres factores desde atención	Partición de equivalencia	Negra	JUnit
CP-16	RF-06, RF-12	Niveles con umbrales ajustados	Valores límite	Negra	JUnit
CP-17	RF-06	Caminos básicos de ClasificadorNivel	Caja blanca: caminos (V(G)=5)	Blanca	JUnit + JaCoCo
CP-18	RF-07	Destinatarios según nivel	Partición de equivalencia	Negra	Postman + JUnit
CP-19	RF-07	No repetir durante 3 min	Valores límite de tiempo	Negra	JUnit (reloj simulado)
CP-20	RF-07	Empeora antes de los 3 min	Transición de estados	Negra	JUnit
CP-21	RF-08	Escalamiento de crítico a 45 s	Valores límite de tiempo	Negra	JUnit + Cucumber
CP-22	RF-08	Escalamiento de alto a 90 s	Valores límite de tiempo	Negra	JUnit (reloj simulado)
CP-23	RF-08	Reconocimiento antes del plazo	Transición de estados	Gris	Postman + JUnit + Cucumber
CP-24	RF-08	Reconocer una alerta ya reconocida	Transición de estados	Negra	MockMvc
CP-25	RF-08	Dos reconocimientos simultáneos	Adivinanza de errores	Gris	JUnit (2 hilos) + H2 + MockMvc con mock
CP-26	RF-08	Decisiones de ServicioEscalamiento	Caja blanca: decisiones	Blanca	JUnit + JaCoCo + PIT
CP-27	RF-09	Variaciones dentro de la banda	Partición de equivalencia	Negra	Postman + JUnit + Cucumber
CP-28	RF-09	Bordes de la banda	Valores límite	Negra	JUnit + Cucumber
CP-29	RF-09, RF-03	Maniobra con canal no confiable	Combinatoria	Negra	JUnit
CP-30	RF-09	Maniobra que nunca se ejecuta	Valores límite de tiempo	Negra	JUnit (reloj simulado)
CP-31	RF-03, RF-07/08, RF-09	Combinaciones pairwise	Pairwise	Negra	JUnit (CSV) + Postman
CP-32	RF-12	Límites de umbrales	Valores límite	Negra	Postman + JUnit + Mockito
CP-33	RF-12	Doble aprobación	Tabla de decisión	Negra	JUnit + Cucumber + Mockito
CP-34	RNF seguridad	Acceso sin autenticar	Partición de equivalencia	Negra	Postman + MockMvc + servidor real
CP-35	RNF seguridad	Permisos por rol	Tabla de decisión	Negra	JUnit (parametrizado)
CP-36	RNF seguridad	Endpoints de QA apagados en producción	Partición de equivalencia	Gris	JUnit (perfil prod)
CP-37	RF-03	Magnitud por nombre en la ruta	Partición de equivalencia	Negra	JUnit + MockMvc
CP-38	RF-08	Reconocer una alerta inexistente	Partición de equivalencia	Negra	MockMvc
CP-39	RF-09	Ciclo de vida de la maniobra	Transición de estados	Negra	JUnit
CP-40	RF-12	Aprobar una solicitud ya aplicada	Transición de estados	Negra	JUnit + Mockito

ID	Req.	Caso	Técnica	Caja	Ejecución
CP-41	Versión	Versión informada	Partición de equivalencia	Negra	MockMvc
UI-01	RF-07/08	Reconocer desde el panel	Caso de uso (CU-03)	Negra	Selenium
UI-02	RF-06	El panel muestra la estimación vigente	Caso de uso (CU-02)	Negra	Selenium
UI-03	RF-03	El panel muestra el aviso de canal	Caso de uso (CU-01)	Negra	Selenium

Tres casos completos como ejemplo:

Campo	CP-07 · Canal que discrepa 2,01 %
Requerimiento	RF-03
Técnica y caja	Tabla de decisión (R4) + valores límite · caja negra
Ejecución	Postman + JUnit + Cucumber
Precondiciones	Base
Datos	A = B = 300,00; C = 306,64 °C
Pasos	Enviar las tres lecturas y consultar GET /api/v1/magnitudes/TEMPERATURA_PRIMARIO
Resultado esperado	C NO_CONFIABLE por DISCREPANCIA; el valor usa A y B; aviso CANAL_EXCLUIDO:C
Criterio de aprobación	Estado de C y valor consolidado
Resultado	Aprobado (09/11/2026)

Campo	CP-21 · Escalamiento de crítico a 45 s
Requerimiento	RF-08
Técnica y caja	Valores límite de tiempo · caja negra
Ejecución	JUnit + Cucumber
Precondiciones	Alerta CRITICO sin reconocer en T0
Datos	Reloj en 44, 45 y 46 s
Pasos	Avanzar el reloj y consultar estado y destinatarios
Resultado esperado	44 s sin escalar; 45 s escala al supervisor de seguridad operacional; 46 s no se duplica
Criterio de aprobación	Un único escalamiento a los 45 s
Resultado	Aprobado (09/11/2026)

Campo	CP-33 · Doble aprobación
Requerimiento	RF-12
Técnica y caja	Tabla de decisión · caja negra
Ejecución	JUnit + Cucumber + Mockito
Precondiciones	Solicitud de alto 35 % creada por el supervisor de SO
Datos	Sin aprobar; aprueba el mismo usuario; aprueba el jefe de turno
Pasos	Aplicar cada variante y consultar umbrales
Resultado esperado	Sin aprobar sigue en 40 %. Si aprueba el mismo usuario, 409 (por la API, 403 por rol: ATA-12). Con la aprobación del jefe de turno pasa a 35 %
Criterio de aprobación	Umbral vigente en cada variante
Resultado	Aprobado (09/11/2026)

Cuando la salida es una estimación no esperamos un número exacto sino un nivel. Por eso los casos de RF-06 trabajan con la probabilidad inyectada y verifican el nivel y los factores, no el cálculo del modelo.

9. Criterios de aceptación y BDD

Criterio de aceptación	Casos
Un canal que discrepa más del 2 % se deja de usar y se avisa; uno que discrepa exactamente el 2 % se sigue usando	CP-07, CP-08
Una alerta crítica sin reconocer escala al supervisor de seguridad operacional a los 45 s, y reconocerla antes lo evita	CP-21, CP-23
Durante una maniobra, las variaciones esperables generan una sola alerta informativa y lo que sale de la banda se alerta normal	CP-27, CP-28
Un umbral solo cambia con la aprobación de un segundo rol	CP-33

Los criterios están escritos en cuatro features en español (`redundancia`, `escalamiento`, `maniobras` y `umbrales`), con 11 escenarios en total. Cada escenario lleva un comentario con el ID del caso. Los pasos usan el contexto de Spring con el perfil qa, así que corren contra el SISTEMA completo. Por ejemplo:

```
# language: es
Característica: Escalamiento de alertas críticas
  Para que una alerta crítica no quede sin atender
  como supervisor de seguridad operacional
  quiero recibirla si nadie la reconoce en 45 segundos

  Antecedentes:
    Dado que el sistema está en la precondición base

  # CP-21 · Valores límite de tiempo
  Esquema del escenario: A los <segundos> s sin reconocer la alerta queda <estado>
    Dado una alerta crítica emitida al operador de reactor
    Cuando pasan <segundos> segundos sin reconocimiento
    Entonces el estado de la alerta es <estado>
    Y la alerta tiene <destinatarios> destinatarios

    Ejemplos:
      | segundos | estado   | destinatarios |
      | 44       | EMITIDA  | 2             |
      | 45       | ESCALADA | 3             |
      | 46       | ESCALADA | 3             |
```

10. Automatización, TDD, API e interfaz

Automatizamos todo lo que tiene un resultado verificable sin juicio humano: reglas de dominio, servicios, API e interfaz. Usamos JUnit 5 con pruebas parametrizadas, Spring Boot Test y MockMvc, Cucumber para los criterios de aceptación, Selenium para el panel, JaCoCo para la cobertura y PIT para la mutación. Todo corre con `mvn verify` en un clon limpio, y cada clase de prueba tiene un comentario con su propósito y el ID de los casos que cubre. Cada test lleva el ID en el nombre que muestra JUnit (CP-xx, UI-xx, o INF-xx para las dos pruebas del reloj y de los endpoints de QA, que prueban la infraestructura de pruebas y no el SISTEMA).

El Makefile tiene un comando por tipo de prueba (`make unitarias`, `api`, `integracion`, `bdd`, `ui`), uno para mutación (`make pit`) y uno que corre todo y guarda la salida como evidencia (`make evidencias`).

10.1 TDD

Aplicamos TDD en ValidadorRedundancia y ServicioEscalamiento, que son reglas precisas con bordes numéricos y de tiempo. Hicimos tres ciclos en cada clase: primero el test que falla, después el mínimo código para que pase. La salida de cada ejecución en rojo y en verde está en `docs/evidencias/tdd/` (doce archivos). No dejamos un commit por etapa, como decía la versión anterior de este informe: la evidencia del ciclo es esa salida.

10.2 API y Postman

La colección de Postman tiene una carpeta por requerimiento, un entorno con `baseUrl` y usuarios por rol, y scripts que verifican código HTTP y cuerpo de la respuesta. Cada request que lo necesita llama antes a `/qa/reset`. La corremos completa con newman (`make postman`), que levanta el SISTEMA si no está arriba: 97 requests, 156 aserciones, 0 fallos. La salida queda en `docs/evidencias/postman/`.

Las mismas reglas de la API también se prueban con MockMvc. Postman encontró algo que MockMvc no veía: sin credenciales, el servidor real respondía 302 al login en vez de 401 (ATA-16). MockMvc no hace el reenvío interno a `/error` que causaba el problema. Desde entonces hay pruebas contra el servidor real (`SeguridadServidorRealTest`).

10.3 Interfaz con Selenium

El panel se prueba con Selenium usando Page Objects (página de login y página del panel). Los elementos que usan las pruebas tienen un atributo `data-test`, y las esperas son por condición, nunca fijas. En `mvn verify` corre con HtmlUnit, sin navegador; con `make ui-chrome` corre en Chrome, y `make ui-demo` deja el navegador abierto al final de cada prueba para la exposición. Las capturas de la ejecución en Chrome están en `docs/evidencias/selenium/`. Al revisarlas apareció un defecto que las pruebas no detectaban: los estados de los tres canales salían pegados en un renglón (ATA-18). Agregamos el test que lo muestra, lo vimos fallar y después lo corregimos.

10.4 Mocks

La mayoría de las pruebas usan el SISTEMA real con H2 y el reloj de prueba, sin mocks. Usamos Mockito en dos lugares donde un mock aporta algo que la prueba real no da. En `ServicioUmbralesTest` el repositorio y el validador son mocks, para comprobar que, cuando una regla corta, el servicio no guarda ni cambia nada (CP-32, CP-33, CP-40). En `ConflictoConcurrenteApiTest` el servicio de alertas es un mock que lanza el choque de versión de JPA: así el 409 de CP-25 se prueba sin depender de que dos hilos se crucen.

10.5 Mutation testing con PIT

PIT se corre sobre el paquete de dominio con los mutadores por defecto más `INLINE_CONSTS`. Los mutadores por defecto no cambian constantes; sin `INLINE_CONSTS` no aparecen mutantes como “45 s pasa a 46 s”. Por la misma razón, los límites están como literales o campos de instancia: PIT no muta los inicializadores `static final`. Nos interesan sobre todo los mutantes que cambian bordes, porque son los defectos más probables en este SISTEMA:

Mutante	Dónde	Caso que lo mata	Resultado
>= pasa a > en 15, 40 o 75 %	ClasificadorNivel	CP-13	Muerto
> pasa a >= en el 2 %	ValidadorRedundancia	CP-08	Muerto
45 s pasa a 46 s	ServicioEscalamiento	CP-21	Muerto

Mutante	Dónde	Caso que lo mata	Resultado
90 s pasa a 91 s	ServicioEscalamiento	CP-22	Muerto
180 s pasa a 181 s	PoliticaRepeticion	CP-19	Muerto
Se elimina la condición “ya reconocida”	ServicioEscalamiento	CP-24	Muerto
30 % pasa a 31 % en el límite del umbral alto	ValidadorUmbrales	CP-32	Muerto

Resultado: 162 mutantes, 159 muertos (mutation score 98 %). Sobreviven 3, y los tres son equivalentes: dos en la regla “alto menor que crítico” de ValidadorUmbrales, que nunca puede fallar (ATA-11), y uno en SenalCanal, un indicador que el validador nunca lee para un canal sin datos porque antes corta la regla 1. En el camino agregamos cinco casos por mutantes que sobrevivían (por ejemplo, la mediana con tres valores distintos y el redondeo a dos decimales). El detalle está en `docs/pit-analisis.md`.

11. Defectos, Definition of Done y métricas

No tuvimos Jira. Los defectos se registran en `docs/defectos.md` y en la hoja Defectos de la planilla, con versión, caso, pasos, resultado esperado y observado, evidencia, severidad y prioridad. La severidad es crítica si puede llevar al operador a una mala decisión, alta si demora o duplica una alerta, media si es un error sin efecto directo y baja si es cosmético. El flujo es Nuevo, Confirmado, En corrección, Corregido, Verificado y Cerrado. Si una prueba sigue fallando al entregar, es porque evidencia un defecto registrado, y lo indicamos en su nombre. Los defectos corregidos también quedan citados en el nombre del test que los evita (por ejemplo, “[defecto ATA-16, corregido]”).

Tipo	Cantidad	Claves	Estado
Defectos del SISTEMA	3	ATA-16 (alta), ATA-17 (media), ATA-18 (baja)	Cerrados, con prueba de regresión
Defectos de especificación	12	ATA-1 a ATA-12	Confirmados; se aplica la decisión de la sección 5
Defectos del diseño de pruebas	3	ATA-13 a ATA-15	Cerrados (planilla corregida)

Ejemplo de reporte de un defecto de especificación:

Campo	Contenido
Clave y resumen	ATA-1 · RF-03 no define contra qué ni sobre qué rango se mide el 2 %
Tipo	Defecto de especificación
Requerimiento y caso	RF-03 · CP-07, CP-08
Descripción	El requerimiento dice “discrepa más del 2 % del rango respecto de los otros”. Con tres canales no queda claro si se compara contra la mediana, el promedio o cada par, ni si el rango es el físico o el del instrumento.
Impacto	Dos implementaciones correctas según el texto marcan canales distintos como no confiables, y eso cambia la estimación.
Severidad y prioridad	Alta · Alta
Estado	Confirmado; se pidió aclaración a la cátedra. Mientras tanto se implementa la decisión A1/A2.

Ejemplo de reporte de un defecto del SISTEMA:

Campo	Contenido
Clave y resumen	ATA-16 · Sin credenciales, la API responde 302 al login en vez de 401
Tipo	Defecto del SISTEMA (seguridad)
Requerimiento y caso	RNF seguridad · CP-34, CP-35
Pasos	Levantar el SISTEMA con el perfil qa y llamar GET /api/v1/alertas sin credenciales
Esperado	401 con el error en JSON
Observado	302 con Location: /login. Con MockMvc el caso pasaba
Causa	La API respondía con sendError; el servidor reenviaba el error a /error, lo atendía la cadena de seguridad del panel y redirigía al login
Evidencia	Colección de Postman; SeguridadServidorRealTest
Severidad y prioridad	Alta · Alta
Estado	Cerrado el 31/10/2026. La regresión corre en mvn verify

Consideramos terminado un caso cuando está diseñado y trazado, lo revisó otro integrante, se ejecutó con evidencia, sus defectos están registrados y el build está en verde.

Indicador	Umbral	Resultado
Casos ejecutados (aprobados, fallidos, bloqueados)	100 % ejecutados, 0 bloqueados	44 de 44 ejecutados; 44 aprobados
Defectos abiertos por severidad	0 críticos	0 críticos; ningún defecto del SISTEMA abierto
Cobertura de sentencias y ramas (JaCoCo) en dominio	≥ 80 % y ≥ 70 %	99,6 % y 99,0 %
Mutation score (PIT) en dominio	≥ 75 %	98 %
Requerimientos del alcance con casos ejecutados	100 %	100 %

La cobertura dice qué código se ejecutó, no si las verificaciones son buenas; por eso la miramos junto con la mutación. Los mutantes que sobreviven se analizan y, si muestran un caso faltante, lo agregamos.

12. Informe final

Resultado	Valor
Casos aprobados / fallidos / bloqueados	44 / 0 / 0 (ejecutados el 09/11/2026)
Pruebas automatizadas	307 en `mvn verify`, todas en verde, más 97 requests de Postman
Defectos por severidad	Del SISTEMA: 1 alta, 1 media y 1 baja, los tres cerrados. De especificación: 4 altas, 6 medias y 2 bajas, resueltas con una decisión del equipo
Cobertura JaCoCo (sentencias / ramas)	Dominio 99,6 % / 99,0 %. Todo el SISTEMA 98,5 % / 95,0 %
Mutation score PIT	98 % (159 de 162; los 3 vivos son equivalentes)

Lo más importante que encontramos no estuvo en el código sino en el enunciado: doce puntos donde el requerimiento admite más de una lectura, cuatro de ellos con severidad alta (ATA-1, ATA-3, ATA-5, ATA-8). Los tres defectos del SISTEMA se encontraron con pruebas distintas de las que los “tenían” que encontrar: el 302 apareció con Postman y no con MockMvc, la base compartida apareció como un test que fallaba a veces, y el problema del panel apareció mirando las capturas, no en las aserciones. Eso confirma que conviene combinar niveles y herramientas.

Riesgos residuales:

- El comportamiento del modelo real, que no pudimos probar.
- Los requerimientos que quedaron fuera del alcance (RF-04/05, RF-10, RF-11).
- Las ambigüedades de la sección 5 que la cátedra no llegue a aclarar: si la respuesta difiere de nuestra decisión, cambian los casos que la verifican.
- La concurrencia se probó con dos usuarios a la vez (CP-25), no con carga real.

Limitaciones:

- El SISTEMA es una implementación reducida hecha por nosotros, no el ATALAYA real.
- Los datos son sintéticos.
- El mismo equipo desarrolla y prueba, lo que resta independencia.

Recomendación de liberación: recomendamos liberar la versión 3.1.0 para el alcance probado. Se cumplen todos los criterios de salida de la sección 11 y no queda ningún defecto del SISTEMA abierto. La condición es que la cátedra confirme las decisiones de severidad alta (ATA-1, ATA-3, ATA-5 y ATA-8), porque cambian qué canal se usa, cuándo se repite una alerta y cuándo deja de agruparse una maniobra.

Como mejora queda probar los requerimientos excluidos, sumar pruebas de carga para la concurrencia y evaluar el modelo real con datos de transitorios y arranques.

13. Uso de inteligencia artificial

El 26/10/2026 consultamos al profesor por el uso de IA. Nos respondió que puede ser una herramienta de apoyo, siempre que podamos explicar y justificar el código, su funcionamiento y las decisiones que tomamos, y que no reemplace el razonamiento. La usamos en ese sentido. Las decisiones de la sección 5, el diseño de los casos y el análisis de los defectos los discutimos como equipo, y todo lo que se hizo con ayuda de la IA lo revisamos y lo ejecutamos antes de darlo por bueno. Llevamos además un registro interno de cada uso.

Herramienta	Para qué	Qué verificamos
Claude Code	Apoyo en la implementación del SISTEMA reducido, a partir de nuestras decisiones de la sección 5	Que cada regla y cada valor límite coincidiera con el enunciado y con la sección 5; que `mvn verify` pasara en un clon limpio
Claude Code	Apoyo en la automatización de los casos (JUnit, MockMvc, Cucumber, Selenium), la colección de Postman y los comandos del Makefile	Que cada test tuviera su caso en la planilla y su ID; que los mutantes vivos fueran equivalentes; que la colección corriera completa
Claude Code	Revisión de consistencia entre el informe, la planilla y el registro de defectos, y de la redacción de esta versión	Que cada resultado tuviera su evidencia en `docs/evidencias/` y que los números coincidieran con los reportes

14. Entrega

Se entrega VV-Q2-2026-TN-GrupoE-ATALAYA.zip con el proyecto Maven (código del SISTEMA y de las pruebas), la colección y el entorno de Postman, los reportes de JaCoCo, PIT y Cucumber, la planilla de casos y trazabilidad, el registro de defectos, las evidencias y este informe. El ZIP se arma con `make entrega`, que antes corre la suite completa y PIT y no arma nada si algo falla.

VV-Q2-2026-TN-GrupoE-ATALAYA/	
pom.xml, mvnw, .mvn/	Proyecto Maven con su wrapper (no hace falta tener Maven)
src/main/java/…	SISTEMA

VV-Q2-2026-TN-GrupoE-ATALAYA/	
src/test/java/…	Pruebas JUnit, MockMvc, Mockito, Cucumber y Selenium
src/test/resources/features/	Escenarios .feature
postman/	Colección y entorno exportados
reportes/	JaCoCo, PIT y Cucumber
docs/	Este informe, planilla de casos, registro de defectos, análisis de PIT y evidencias
Makefile, scripts/, package.json, README.md	Comandos para correr cada tipo de prueba, newman y la entrega